# CSA0608 – DESIGN AND ANALYSIS OF ALGORITHMS

## DEBUGGING AND OPTIMISATION OF A SECURITY MONITORING MODULE

**Student Name:** Aswini S

**Register Number:** 192511077

**Course:** CSA06 – Design and Analysis of Algorithms

**Assignment:** Nine-Defect Debugging and Optimisation Task

**Language:** Python

# ABSTRACT

This report investigates a security-monitoring module containing four functions: alert sorting, blocklist membership checking, signature searching in a log, and finding the closest pair of login locations. The supplied implementation contains nine defects. The defects are reproduced using the assignment test data, their causes are explained, and each function is corrected and tested. The corrected implementation is then optimised for repeated lookups and large input batches. Operation counts and measured experiments are used to compare the original and improved approaches.

## 1. OBJECTIVES

- Reproduce the incorrect outputs and identify all nine defects.
- Explain the root cause of every defect and trace the corrected logic.
- Test normal cases and boundary cases, including empty inputs and equal values.
- Optimise the functions without changing their intended outputs.
- Analyse time complexity and measure sorting comparisons and closest-pair distance computations.
- Provide production-quality Python code and a repeatable test harness.

## 2. GIVEN TEST DATA AND EXPECTED BEHAVIOUR

The module contains four functions. The intended behaviour is: sort alerts by descending severity while preserving the order of equal severities; return True if an IP address is in the blocklist; return every starting index of a signature in the log; and return the pair of distinct login events having the minimum Euclidean distance.

| Input | Value |
|---|---|
| alerts | [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)] |
| blocklist | ["10.0.0.5", "10.0.0.9", "172.16.4.2"] |
| queries | ["10.0.0.9", "10.0.0.5", "8.8.8.8"] |
| log / pattern | "FAILFAIXFAIL" / "FAIL" |
| events | [(2,3), (12,30), (40,50), (5,1), (12,10), (3,4)] |

For closest_logins, the coordinates are measured in kilometres. The intended distance is the Euclidean distance between two different events.

## PART (a) – REPRODUCE AND IDENTIFY THE DEFECTS (10 MARKS)

### 3. Expected output versus actual output

| Function | Expected output | Actual output from supplied code | Observation |
|---|---|---|---|
| sort_alerts | [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)] | [('A4', 9), ('A3', 5), ('A3', 5), ('A5', 1), ('A5', 1)] | First element is overwritten; records are lost and order is wrong. |
| is_blocked | [True, True, False] | [False, False, False] | Every query returns False, including listed IP addresses. |
| find_signature | [0, 8] | [0, 4] | Misses final match at index 8 and reports false match at index 4. |
| closest_logins | Pair (0, 5), distance $\sqrt{2} \approx 1.4142$ | Pair (0, 2), distance 60.4401 | Returns a farthest pair, and compares self/reversed pairs. |

### 4. All nine defects and faulty statements

| No. | Function | Faulty line / statement | Nature of defect |
|---|---|---|---|
| 1 | sort_alerts | while j > 0 | Boundary condition excludes index 0. The first alert is never compared or shifted. |

| | | | |
|---|---|---|---|
| 2 | sort_alerts | arr[j] = key | The key is inserted one position too early. Correct insertion position is arr[j + 1]. |
| 3 | is_blocked | if blocklist[i] == ip: return False | A matching address is incorrectly reported as not blocked. It must return True. |
| 4 | is_blocked | else: return False (inside the loop) | The function exits after the first non-match, so later blocklist entries are never checked. |
| 5 | find_signature | for i in range(n - m) | The final valid alignment is excluded. The range must include n-m, i.e. range(n-m+1). |
| 6 | find_signature | if j == m - 1: hits.append(i) | A mismatch on the final character still leaves j == m-1, causing a false positive. A full-match flag or KMP is required. |
| 7 | closest_logins | if d > min_dist | The condition finds the maximum distance, not the minimum distance. Initialise best to infinity and minimise squared distance. |
| 8 | closest_logins | for j in range(len(events)) | Includes self-pairs and both (i,j) and (j,i), causing redundant work. Use j from i+1. |
| 9 | closest_logins | math.sqrt(...) inside every pair iteration | Square roots are unnecessary for comparisons. Compare squared distances and take one square root after finding the best pair. |

The nine defects are counted as two in sort_alerts, two in is_blocked, two in find_signature, and three in closest_logins.

# PART (b) – ROOT CAUSES AND SYSTEMATIC DEBUGGING (12 MARKS)

## 5. Root-cause explanation and correction

**sort_alerts:** The loop condition j > 0 stops before comparing the first element. The final assignment arr[j] = key then overwrites the element at j instead of inserting after the last larger element. The corrected insertion-sort form uses j >= 0 and arr[j+1] = key. For equal severities, the comparison remains strict (<), so equal items keep their original order.

**is_blocked:** The function returns False for both a match and a mismatch. In addition, the else return is inside the loop, so only the first item is examined. The corrected function returns True on a match and False only after the loop finishes. For production repeated queries, the list is converted to a set once.

**find_signature:** A log of length n and pattern of length m has n-m+1 possible starting positions. The original range omits the last one. Its inner-loop test also confuses 'last character compared' with 'all characters matched'. KMP avoids this ambiguity by using a prefix table and only records a hit after the entire pattern is matched.

**closest_logins:** The original function updates when a distance is larger, so it returns the farthest pair. It also visits self-pairs and duplicate reversed pairs. Comparing squared distances avoids repeated square roots. The optimised version sorts by x-coordinate and stops scanning a later point when its x-gap squared is already greater than the best squared distance.

## 6. Step-by-step insertion trace

| Step | List after insertion (ID:severity) | Explanation |
|---|---|---|
| Start | A1:3, A2:9, A3:5, A4:9, A5:1 | Initial order |
| Insert A2:9 | A2:9, A1:3, A3:5, A4:9, A5:1 | 9 moves before 3 |
| Insert A3:5 | A2:9, A3:5, A1:3, A4:9, A5:1 | 5 is inserted after 9 |
| Insert A4:9 | A2:9, A4:9, A3:5, A1:3, A5:1 | Equal 9 remains after earlier A2 |
| Insert A5:1 | A2:9, A4:9, A3:5, A1:3, A5:1 | 1 remains at the end |

Correct stable result: A2, A4, A3, A1, A5. The supplied defective code instead produces A5, A2, A4, A4, A5, showing that A1 and A3 are lost and A4/A5 are duplicated.

## 7. Signature alignment trace

| Start index | Compared text | Result | Correct action |
|---|---|---|---|
| 0 | FAIL | Full match | Record index 0 |
| 1 | A... | Mismatch at first character | No match |
| 2 | I... | Mismatch at first character | No match |
| 3 | L... | Mismatch at first character | No match |
| 4 | FAIX | Mismatch at final character (X ≠ L) | Original code falsely records 4 |
| 5–7 | A..., I..., X... | Mismatch | No match |
| 8 | FAIL | Full match at final legal start | Record index 8 |

There are 12 log characters and 4 pattern characters, so valid start positions are 0 through 8 inclusive. The correct result is [0, 8].

## 8. Closest-login calculation

The closest events are event 0 = (2,3) and event 5 = (3,4). Their squared distance is $(2-3)^2 + (3-4)^2 = 1 + 1 = 2$. Therefore the Euclidean distance is $\sqrt{2} = 1.4142$ km. The original code instead selects a farthest pair because it uses a greater-than comparison.

## 9. Re-testing after corrections and boundary tests

| Test case | Input | Expected result | Corrected result |
|---|---|---|---|
| Main alert list | Given five alerts | [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)] | [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)] |
| Blocklist queries | Three given IPs | [True, True, False] | [True, True, False] |
| Main signature | FAILFAIXFAIL / FAIL | [0, 8] | [0, 8] |
| Main closest pair | Six given coordinates | (0,5), √2 | (0, 5), 1.4142 |
| Empty blocklist | []; query 1.2.3.4 | False | False |
| Pattern at final position | ABCXYZ / XYZ | [3] | [3] |
| Pattern longer than log | ABC / LONGER | [] | [] |
| Overlapping signatures | AAAAA / AAA | [0,1,2] | [0, 1, 2] |
| Equal severities | X:5, Y:5, Z:3 | X:5, Y:5, Z:3 | [('X', 5), ('Y', 5), ('Z', 3)] |
| Zero login events | [] | (None, infinity) | (None, inf) |
| One login event | [(4,7)] | (None, infinity) | (None, inf) |

The corrected tests pass. The chosen empty-pattern policy is to return an empty list; this is documented in the function and avoids treating every string boundary as a signature occurrence.

# PART (c) – OPTIMISE THE CORRECTED MODULE (10 MARKS)

## 10. Optimisation decisions

| Function | Corrected basic approach | Production optimisation | Reason |
|---|---|---|---|
| sort_alerts | Stable insertion sort | Use Python's stable Timsort via list.sort(key=..., reverse=True) | Insertion sort is good for tiny/nearly sorted lists, but 50,000 alerts favour adaptive O(n log n) sorting. |
| is_blocked | Sequential list search | Build a set once and use membership | Repeated list scans cost O(qb); set construction costs O(b), then q average O(1) checks. |
| find_signature | Brute-force alignment scan | Knuth-Morris-Pratt (KMP) | Prefix table prevents rechecking matched characters; O(n+m) time. |
| closest_logins | All ordered pairs | Sort by x; scan forward with x-gap pruning; squared distances | Each unordered pair is considered at most once; no self-pairs; no square root per candidate; many pairs are pruned. |

Insertion sort remains useful as a teaching algorithm and for very small or almost perfectly ordered inputs. However, for a production workload of approximately 50,000 alerts, Python's built-in stable sort is preferred because it is adaptive to existing order and has O(n log n) worst-case time.

The closest-pair pruning is an exact branch-and-bound improvement. Its worst-case time remains $O(n^2)$ when many points lie within the current x-distance bound; its observed work can be much lower on spatially spread data. It preserves the original event indices and uses lexicographic tie-breaking so results are deterministic.

# PART (d) – ANALYSE BEFORE AND AFTER (10 MARKS)

## 11. Operation counts and complexity

| Function | Original / corrected basic operations | Before complexity | Optimised complexity |
|---|---|---|---|
| sort_alerts | Insertion sort: up to n(n−1)/2 key comparisons and shifts; best case n−1 comparisons. | Best O(n); average/worst O($n^2$). | Timsort: O(n) best on ordered data; O(n log n) average/worst. |
| is_blocked | Sequential search: at most b equality checks per query. | O(b) per query; O(qb) for q queries. | Set build O(b); average O(1) per query, O(b+q) total expected. |
| find_signature | Brute force: at most (n−m+1)m character comparisons. | O(nm) time; O(1) auxiliary space excluding hits. | KMP: O(m) prefix work + O(n) scan; O(n+m) time and O(m) auxiliary space. |
| closest_logins | Original: $n^2$ ordered pair distance calculations, including n self-pairs and duplicate reversals. | O($n^2$) time; O(1) auxiliary space. | Sort O(n log n) plus pruned scan; worst O($n^2$), often fewer distance calculations; O(n) auxiliary space. |

Symbols: n = number of alerts or log characters/events as appropriate; b = blocklist size; q = number of IP queries; m = pattern length. The output list or result storage is not counted as auxiliary working space.

## 12. Closest-pair distance computation measurements

The following experiment uses deterministic, uniformly distributed two-dimensional points in a 10,000 × 10,000 coordinate area. Both algorithms are checked to return the same closest pair and distance. A distance computation means evaluating the squared Euclidean distance for one candidate pair; square-root calls are not counted as pair computations.

| Events (n) | Original brute-force calculations | Optimised candidate pairs | Reduction |
|---|---|---|---|
| 1000 | 1,000,000 | 878 | 1,139.0x |
| 4000 | 16,000,000 | 1630 | 9,816.0x |
| 8000 | 64,000,000 | 8974 | 7,131.7x |

The supplied brute-force implementation visits every ordered pair, so it performs $n^2$ distance calculations, including n self-pairs and both directions of each distinct pair. A corrected but unpruned i<j baseline would perform n(n−1)/2. The optimised scan considers only i<j and stops when the x-gap alone proves that later points cannot improve the current best distance.

## 13. Insertion-sort comparison measurements

The comparison counter below counts each key-to-alert severity comparison, including the final failed comparison when the while-loop reaches an element. Nearly sorted inputs start in descending severity order and receive sparse adjacent swaps. Random inputs use random severities.

| Alerts (n) | Nearly sorted comparisons | Random-order comparisons |
|---|---|---|
| 1000 | 1004 | 244492 |
| 4000 | 4002 | 3917847 |
| 8000 | 8001 | 15997186 |

The near-sorted case requires far fewer comparisons because insertion sort performs work proportional to the number of inversions plus a small linear scan. Random data generates many inversions, so comparisons grow quadratically. This supports the theoretical analysis: insertion sort can be excellent for nearly sorted data, but random or large workloads can make it expensive. Timsort is selected for production because it detects existing runs and retains an O(n log n) worst-case bound.

## 14. Interpretation of results

- The brute-force closest-pair count grows exactly quadratically as n(n−1)/2.

- The pruned closest-pair count is data-dependent and is expected to be much smaller for well-spread random points; the worst case is still quadratic.
- Nearly sorted alerts need substantially fewer insertion-sort comparisons than random alerts.
- Measured behaviour agrees with the operation-count analysis. Timing may vary by computer, Python version, and system load, so operation counts are the more reproducible measure.

# PART (e) – PRODUCTION-QUALITY CODE (8 MARKS)

## 15. Final optimised Python module

```
"""Security monitoring algorithms: alerts, blocklists, signatures and logins."""
import math


def sort_alerts(alerts):
    """Sort (alert_id, severity) records by descending severity, stably.

    The input list is sorted in place and returned.
    """
    alerts.sort(key=lambda alert: alert[1], reverse=True)
    return alerts


def build_blocklist_index(blocklist):
    """Convert a blocklist to a set once for repeated membership queries."""
    return set(blocklist)


def is_blocked(blocklist, ip):
    """Return True if ip is blocked; use a set for average O(1) lookup."""
    return ip in blocklist


def find_signature(log, pattern):
    """Return all match positions, including overlaps, using KMP.

    An empty pattern returns [] by design.
    """
    if not pattern:
        return []

    # LPS[i] is the longest proper prefix that is also a suffix
    # for pattern[0:i+1].
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1

    hits = []
    i = j = 0
    while i < len(log):
        if log[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                hits.append(i - j)
                # Continue from the longest reusable suffix (overlaps).
                j = lps[j - 1]
        elif j:
            j = lps[j - 1]
        else:
            i += 1
```

```
    return hits


def closest_logins(events):
    """Return (closest_original_index_pair, Euclidean_distance).

    Events are (x, y) coordinate pairs. Ties use the lexicographically
    smallest original index pair. For fewer than two events, return
    (None, math.inf).
    """
    n = len(events)
    if n < 2:
        return None, math.inf

    # Keep original indices while sorting spatially by x then y.
    ordered = sorted((x, y, idx) for idx, (x, y) in enumerate(events))
    best_sq = math.inf
    best_pair = None

    for i in range(n - 1):
        x1, y1, idx1 = ordered[i]
        for j in range(i + 1, n):
            x2, y2, idx2 = ordered[j]
            dx = x2 - x1
            dx_sq = dx * dx

            # Sorted x values mean all later points have at least this x-gap.
            if dx_sq > best_sq:
                break

            dy = y2 - y1
            distance_sq = dx_sq + dy * dy
            pair = tuple(sorted((idx1, idx2)))

            if (distance_sq < best_sq or
                    (distance_sq == best_sq and
                     (best_pair is None or pair < best_pair))):
                best_sq = distance_sq
                best_pair = pair

    # One square root after the minimum squared distance is known.
    return best_pair, math.sqrt(best_sq)


def run_tests():
    """Run main and boundary tests for every public function."""
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    assert sort_alerts(alerts) == [
        ("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)
    ]
    assert sort_alerts([("X", 5), ("Y", 5), ("Z", 3)]) == [
        ("X", 5), ("Y", 5), ("Z", 3)
    ]

    blocked = build_blocklist_index(
        ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
    )
    assert [is_blocked(blocked, ip) for ip in
            ["10.0.0.9", "10.0.0.5", "8.8.8.8"]] == [True, True, False]
    assert is_blocked(build_blocklist_index([]), "1.2.3.4") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCXYZ", "XYZ") == [3]       # final position
    assert find_signature("ABC", "LONGER") == []        # pattern too long
    assert find_signature("AAAAA", "AAA") == [0, 1, 2]  # overlapping
    assert find_signature("ABC", "") == []

    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    pair, distance = closest_logins(events)
    assert pair == (0, 5) and math.isclose(distance, math.sqrt(2))
    assert closest_logins([]) == (None, math.inf)
    assert closest_logins([(4, 7)]) == (None, math.inf)

    # Duplicate coordinates: deterministic lexicographic tie-breaking.
    assert closest_logins([(0, 0), (1, 1), (0, 0)])[0] == (0, 2)
    print("All tests passed.")
```

```
if __name__ == "__main__":
    run_tests()
```

## 16. Pseudocode for the optimised functions

```
ALGORITHM SORT_ALERTS(alerts)
    SORT alerts by severity in descending order using a stable sort
    RETURN alerts
END

ALGORITHM BUILD_BLOCKLIST_INDEX(blocklist)
    RETURN HASH_SET(blocklist)
END

ALGORITHM IS_BLOCKED(index, ip)
    RETURN ip EXISTS IN index
END

ALGORITHM FIND_SIGNATURE(log, pattern)
    IF pattern is empty THEN RETURN empty list
    BUILD the KMP longest-prefix-suffix (LPS) table
    i ← 0; j ← 0; hits ← empty list
    WHILE i < length(log)
        IF log[i] = pattern[j]
            i ← i + 1; j ← j + 1
            IF j = length(pattern)
                APPEND i-j to hits
                j ← LPS[j-1]
        ELSE IF j > 0
            j ← LPS[j-1]
        ELSE
            i ← i + 1
    RETURN hits
END

ALGORITHM CLOSEST_LOGINS(events)
    IF fewer than 2 events THEN RETURN (None, infinity)
    SORT (x, y, original_index) by x then y
    best_squared ← infinity; best_pair ← None
    FOR each point i from left to right
        FOR each later point j
            dx_squared ← (xj-xi)^2
            IF dx_squared > best_squared THEN BREAK
            distance_squared ← dx_squared + (yj-yi)^2
            UPDATE best pair if distance is smaller
            ON equal distance, keep lexicographically smaller index pair
    RETURN (best_pair, square_root(best_squared))
END
```

## 17. Test harness output

```
=== RESTART: C:/Users/aswin/AppData/Local/Programs/Python/Pythor
SORTED ALERTS:
[('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

BLOCKLIST RESULTS:
10.0.0.9 : True
10.0.0.5 : True
8.8.8.8 : False

SIGNATURE POSITIONS:
[0, 8]

CLOSEST LOGIN PAIR:
Pair: (0, 5)
Distance: 1.4142 km

ALL TESTS PASSED!
>
```

## 18. GitHub submission procedure

- Create a folder named SecurityMonitoringModule.
- Save the final code as security_monitor.py.
- Run the program in VS Code or a terminal using: python security_monitor.py.
- Confirm that the terminal displays: All tests passed.
- Create a GitHub repository and upload security_monitor.py and this report.
- Open the uploaded file and verify that the code is visible and readable.
- Copy the repository URL and submit that link through Google Classroom.

GitHub repository link: _____

Note: The report and code are prepared here, but a GitHub repository cannot be committed or published without the user's GitHub account action. Paste the repository URL above after uploading.

## 19. CONCLUSION

The supplied module contained nine defects that affected sorting correctness, blocklist decisions, signature matching, and closest-login identification. The debugging process identified each faulty statement and corrected the logic using stable sorting, correct membership semantics, KMP string matching, and a pruned closest-pair scan. Boundary tests confirmed the corrected behaviour for empty inputs, end-position matches, long patterns, overlapping matches, equal severities, and insufficient login events. The complexity analysis and experiments show why stable Timsort, hash-set membership, KMP, and squared-distance pruning are suitable improvements for the stated workload. The final implementation includes docstrings, comments, deterministic tie handling, and an assertion-based test harness.

## 20. REFERENCES

1. CSA06 – Design and Analysis of Algorithms, CO2 assignment question document supplied for this task.

2. Python Software Foundation, Python documentation: list.sort() and sorting techniques (stable sorting).

3. Knuth, D. E., The Art of Computer Programming, Volume 3: Sorting and Searching (string matching and algorithm analysis).

4. Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C., Introduction to Algorithms (algorithm design and complexity analysis).